

תהליכים

מערכות הפעלה

תהליך

- תהליך הוא ריצה של תוכנה במערכת ההפעלה
- אנו רגילים לחשוב על תוכנה כ"עושה משהו"
- תוכנה שמבצעת חישובים
- תוכנה שמשמיעה מוזיקה
- אבל בעצם תוכנה לא עושה כלום – היא קובץ שיושב במחשב
- התקנתי במחשב תוכנה שמשמיעה מוזיקה, האם ברגע שהתקנתי אותה אני אתחיל לשמוע מוזיקה? בוודאי שלא
- מה שבאמת "עושה משהו" זה התהליך
- אפשר לחשוב על תוכנה כ"פוטנציאל" ועל תהליך כ"מימוש הפוטנציאל"

חלוקת משאבים

- במבוא למחשבים הכרנו משאבים שונים של המחשב שתוכנה עושה בהם שימוש, או ליתר דיוק – שתהליך עושה בהם שימוש
- המשאבים כוללים רכיבי זיכרון, בינהם:
 - יחידת הזיכרון, המשמשת לאיחסון נתונים
 - רגיסטרים, רכיבי זיכרון בתוך ה-CPU השומרים "תאי אחסון" לחישובים מידיים, ונתונים חשובים להרצת התהליך
- המודל שתיארנו במבוא למחשבים, מתאר מערכת בה המחשב מריץ תוכנה יחידה המשתשמת בכל משאבי המחשב
- כי שתיארנו במערכת ההפעלה זה לא המצב – יש תהליכים רבים הרצים בכל רגע נתון
- למשל במערכת הפעלה של מחשב אישי, בכל רגע נתון, לרוב רצים מאות תהליכים במקביל

חלוקת משאבים

• יחידת הזיכרון

- יחידת הזיכרון מחולקת לאזורים שונים המכילים סוגים שונים של נתונים לשימוש התהליך. ביניהם:
 - Code Segment – שמירת הקוד של התהליך, כלומר הפקודות שהוא צריך להריץ
 - Data Segment – מיועד לשמירה של משתנים שערכם מאותחל בתחילת התהליך
 - BSS Segment – מיועד לשמירה של משתנים שערכם אינו מאותחל בתחילת התהליך
 - Data ו BSS מהווים את ה"זיכרון הסטטי" – זיכרון שגודלו ידוע מראש (בזמן קומפילציה)
 - Heap Segment – מיועד לשמירה של נתונים "זמניים" – נתונים שיכולים להיווצר ולהימחק במהלך ריצת התוכנית
 - Heap מהווה את ה"זיכרון דינאמי" – זיכרון שגודלו אינו ידוע מראש (יש לו הגבלה מסויימת)
 - ברוב שפות התכנות המודרניות, ערכים מורכבים (כגון מערכים ואובייקטים) נשמרים תמיד בזיכרון הדינאמי
 - Stack Segment – סגמנט המחסנית המיועד לשמירה של ערכים אשר נחוצים לניהול מעבר בין קטעים שונים בתוך הקוד (כגון פונקציות)

חלוקת משאבים

- רגיסטרים
- רכיבי זיכרון בתוך ה-CPU השומרים:
- "תאי אחסון" לחישובים מידיים
- נתונים חשובים להרצת התהליך כגון:
- IP (instruction pointer) – מכיל את הכתובת הבאה לביצוע – דרכו מתבצע ניהול של ריצת פקודות התהליך
- BP (base pointer) SPI (stack pointer) – רגיסטרים המשמשים לעבודה עם סגמט המחסנית
- כתובות של סגמנטים שונים בזיכרון

חלוקת משאבים

- מערכת ההפעלה היא בעצם התוכנה היחידה שרצה במחשב, וחלק מהתפקיד שלה הוא לנהל את התהליכים הרצים ואת הגישה שלהם למשאבים באופן שבו:
- מצד אחד – לכל תהליך תהיה גישה למשאבים הנחוצים לו
- מצד שני – לא יהיה "התנגשויות": תהליך אחד לא יגש למשאבים באופן שיפגע בריצה של תהליך אחר
- כיצד מערכת ההפעלה מבצעת זאת?

חלוקת משאבים

- מערכת ההפעלה נתנה לכל תהליך מקטע של 500byte מהזיכרון – לראשון החל מכתובת 500, ולשני החל מכתובת 1000

- לאחר ריצת התהליכים

- הערך 66 ירשם בכתובת 650:

$$\begin{array}{ccccc} \text{area for process 1} & & \text{ds} & & \text{address} \\ & \underbrace{\hspace{1cm}}_{500} & + & \underbrace{\hspace{1cm}}_{100} & + & \underbrace{\hspace{1cm}}_{50} \end{array}$$

- הערך 88 ירשם בכתובת 1150:

$$\begin{array}{ccccc} \text{area for process 2} & & \text{ds} & & \text{address} \\ & \underbrace{\hspace{1cm}}_{1000} & + & \underbrace{\hspace{1cm}}_{100} & + & \underbrace{\hspace{1cm}}_{50} \end{array}$$

• יחידת הזיכרון

- מערכת ההפעלה מקצה לכל תהליך מקטע ביחידת הזיכרון

- כל תהליך מתייחס למקטע שלו כאילו הוא "כל הזיכרון"

- משמעות הדבר היא שגם אם שני תהליכים מתייחסים לאותה כתובת, זו לא באמת אותה כתובת – כי זו כתובות ביחס לחלק שלהם בזיכרון

- לדוגמא נניח שיש שני תהליכים, כאשר כל אחד מהם מריץ תוכנה, אשר בה:

- data segment נמצא בכתובת 100

- הם שמים בכתובת 50 (ביחס לds) ערך – הראשון את הערך 66, והשני את הערך 88

חלוקת משאבים

- **רגיסטרים**
- את הרגיסטרים לא ניתן לחלק כיוון שהן רכיבים פיזיים במעבד
- למשל, יש רגיסטר IP יחיד
- עבור כל תהליך, מערכת ההפעלה שומרת מבנה נתונים הנקרא בלוק בקרת תהליך השומר מספר נתונים עבור תהליך, ביניהם ערכי רגיסטרים
- מערכת ההפעלה משתמשת בבלוקים אלה כדי לשמור על ערכי הרגיסטרים של התהליכים השונים, ולוודא שהם אינם "נהרסים" ע"י ריצה של תהליכים אחרים

בלוק בקרת תהליך

- בלוק בקרת תהליך (PCB – Process Control Block) הוא מבנה נתונים אשר מערכת ההפעלה שומרת עבור כל תהליך שנמצא בה
- הוא שומר את כל הנתונים הנחוצים להרצה תקינה של התהליך, כגון:
 - מצב התהליך (כגון: בריצה, בהשהייה)
 - מהו איזור הזיכרון המוקצה לתהליך
 - ערכי הרגיסטרים, בינהם:
 - כתובות הסגמנטים השונים
 - כתובת הפקודה הבאה לביצוע

חילוף הקשר

- במהלך ריצה של מערכת ההפעלה, היא צריכה לבצע מעברים בין תהליכים שונים הרצים בה
- דוגמאות:

- 1. רץ תהליך אשר מחשב סכום ספרות של מספר, התהליך רץ באופן הבא:
 - 1. הדפסת הודעה למשתמש להכניס מספר
 - 2. קליטת מספר מהמשתמש
 - 3. חישוב סכום הספרות של המספר
 - 4. הדפסת התוצאה
- בפועל ירוצו שלושה תהליכים – התהליך "הראשי", ה driver של המסך עבור הדפסות, ה driver של המקלדת עבור קליטה
- תהליך הריצה:
 - 1. התהליך הראשי ירוץ עד שתבצע ההדפסה הראשונה
 - 2. ה driver של המסך ירוץ כדי לשנות את התצוגה להכיל את ההודעה
 - 3. התהליך הראשי ירוץ עד שיבקש קלט
 - 4. ה driver של המקלדת ירוץ כדי לקלוט
 - 5. התהליך הראשי ירוץ – יחשב וידפיס
 - 6. ה driver של המסך ירוץ לשנות שוב את המסך

חילוף הקשר

- במהלך ריצה של מערכת ההפעלה, היא צריכה לבצע מעברים בין תהליכים שונים הרצים בה
- דוגמאות:
- 2. יש מחשב אשר אחרי על הצגת של אוסף תמונות בזמן שמתנגת מוזיקת רקע
 - הוא מריץ שני תהליכים במקביל
 - PicShow – תהליך שאחראי על הצגת התמונות
 - Music - תהליך שאחרי על השמעת המוזיקה
 - הערה: בדומה לדוגמא הקודמת גם רצים תהליכי הdriver עבור עזרי הפלט, אבל נתעלם מהם לשם הפשטות
 - שני התהליכים רצים במקביל – לא נרצה שקודם יוצגו כל תמונות ורק לאחר מכן תישמע המוזיקה
- כדי לקבל הרצה במקביל מתבצעת הריצה הבאה:
 - PicShow רץ לזמן קצר
 - Music רץ לזמן קצר
 - PicShow רץ לזמן קצר
 - Music רץ לזמן קצר
 - וכך הלאה
- הזמנים כל כך קצרים והמעבר בין התהליכים כל כך מהיר, שנוצרת "אשליה" שהם רצים במקביל

חילוף הקשר

- במהלך ריצה של מערכת ההפעלה, היא צריכה לבצע מעברים בין תהליכים שונים הרצים בה
- מערכת ההפעלה צריכה לוודא שהמעבר ביניהם יהיה תקין, כלומר כאשר מתבצע מעבר לתהליך מסויים היא תוודא:
- התהליך ימשיך את הריצה שלו מהנקודה בו הפסיק
- כל התהליכים שרצו מהרגע שהתהליך הפסיק את הרגע שהוא חזר לרוץ, לא "הרסו" אותו

חילוף הקשר

- התהליך של מעבר בין תהליכים נקרא חילוף הקשר (Context Switch)
- עושה שימוש במבני ה-PCB
- Context switch בין תהליך א לתהליך ב:
- מערכת ההפעלה מוודאת שכל הנתונים ב-PCB של תהליך א מעודכנים למצב אליו הגיע כאשר מתבצע החילוף
- מערכת ההפעלה קוראת את הנתונים ב-PCB של תהליך ב, ומעדכנת את המערכת (בפרט את הרגיסטרים ב-CPU) להיות במצב בו תהליך ב הפסיק
- ריצה ממשיכה – כיוון שהרגיסטרים מעודכנים ובפרט הרגיסטר IP אשר קובע מה הפקודה הבאה לביצוע, זה יגרום לכך שתהליך ב ימשיך בריצתו

חילוף הקשר

- כדי להבין את התהליך נראה תוכנת C# המבצעת תהליך דומה
- התוכנה תכלול 3 מונים (כל אחד מייצג תהליך)
- בכל שלב התוכנה תגדיל את ערך המונה של התהליך הנוכחי ב1 ותדפיס את ערכו
- לאחר מכן "תטיל מטבע" – אם "יצא עץ", תחליף את ההקשר לתהליך אחר
- התוכנה תכלול משתנה יחיד counter איתו תעבוד (מייצג רגיסטר)
- התוכנה תשמור מערך של מונים עבור כל אחד מהתהליכים (מייצגים את הPCB)
- חילוף הקשר ישמור את הערך ה"רגיסטר" ב"PCB" של התהליך הנוכחי ותטען אליו את הערך ב"PCB" של התהליך הבא

```

static readonly int[] PCB = new int[3]; "PCBs"
static int process = 0; "תהליך" נוכחי
static int counter = 0; "רגיסטר"
0 references
static void Main()
{
    Random r = new();
    Console.ForegroundColor = ConsoleColor.Blue;
    for (int i = 0; i < 20; i++)
    {
        counter++;
        Console.WriteLine($"[process {process}] {counter}");
        if (r.Next(2) == 0)
            ContextSwitch();
    }
}
static readonly ConsoleColor[] colors =
    [ConsoleColor.Blue, ConsoleColor.Red, ConsoleColor.Green];
1 reference
static void ContextSwitch()
{
    PCB[process] = counter;
    process = (process + 1) % 3;
    counter = PCB[process];
    Console.ForegroundColor = colors[process];
}

```

```

[process 0] 1
[process 0] 2
[process 0] 3
[process 1] 1
[process 2] 1
[process 2] 2
[process 2] 3
[process 0] 4
[process 1] 2
[process 1] 3
[process 2] 4
[process 2] 5
[process 0] 5
[process 1] 4
[process 1] 5
[process 2] 6
[process 2] 7
[process 0] 6
[process 0] 7
[process 0] 8

```

הרצת תהליכים

- אנו יכולים לגרום להרצה של תהליכים בשתי דרכים:
 1. בקשה ישירה ממערכת הפעלה, ע"י ממשק משתמש (ממשק גרפי/ממשק פקודות)
 2. הרצת בקשה ע"י תוכנה
- בשלב ראשון נכיר את ממשק הפקודות cmd של windows, ופקודות נפוצות לשימוש בו
- בשלב השני נראה כיצד לכתוב קוד C# המבקש מהמערכת להריץ פקודות